

Índice

1 Laboratorio 1: Recomendador de Videojuegos	1
1.1 Objetivos	2
1.2 Objetivos de implementación	2
1.2.1 Si no conocés Python, tests o Docker	2
1.3 API para gestión de información	2
1.4 Material provisto (kickstarter)	3
1.5 Antes de empezar: conceptos de red	3
1.5.1 Conceptos de calidad de código (para este lab)	3
1.6 Contrato de la API	4
1.6.1 Relación entre esquemas de OpenAPI y estructuras en código	5
1.7 Archivos a completar o modificar	5
1.7.1 Sugerencia de reparto del trabajo (grupos de 4)	6
1.8 Parte 1: Jugando con APIs (curl)	6
1.8.1 Ejemplo 1 — Búsqueda de entidades en Wikidata	6
1.8.2 Ejemplo 2 — Obtener una entidad por ID	6
1.8.3 Ejemplo 3 — Búsqueda con otro término y límite	7
1.8.4 Consignas para que redactes vos	7
1.9 Parte 2: Configuración del entorno, Swagger y primeros tests	7
1.10 Parte 3: API de usuarios	8
1.11 Parte 4: Lista de juegos por usuario	9
1.12 Parte 5: Integración con Wikidata	9
1.12.1 Documentación de la API de Wikidata	9
1.13 Parte 6: Autenticación básica	10
1.13.1 Modelo de datos (referencia)	10
1.13.2 Uso del token y autorización mínima	11
1.13.3 Relación con la aprobación	11
1.14 Parte 7: Evaluación de la API y entrega final	11
1.15 Condiciones de aprobación	12
1.16 Entrega	12
1.16.1 Cómo y cuándo entregar	13
1.17 Estructura e indicaciones para el video	13
1.18 Criterios de evaluación (para autoevaluación)	14
1.18.1 Evaluación del código	14
1.18.2 Evaluación del video	17
1.18.3 Puntajes mínimos para aprobar cada dimensión	18
1.19 Recomendaciones para el laboratorio	18
1.20 Mejoras opcionales (puntos estrella)	19
1.21 Cómo ejecutar	19
1.22 Recursos por tema	20
1.23 Material de lectura adicional	20
1.24 Seguridad y producción (a nivel conceptual)	20
1.25 Para después	20

1 Laboratorio 1: Recomendador de Videojuegos

Cátedra de Redes y Sistemas Distribuidos — 2026

Revisión 2026 — Versión 2.0.0

1.1 Objetivos

- Comprender qué es una API REST y cómo se diseña y consume.
- Implementar una API con Flask que cumpla un contrato definido (OpenAPI).
- Consumir una API externa (Wikidata) e integrar sus datos en tu aplicación.
- Usar herramientas de desarrollo y pruebas: curl, pytest, Docker, entornos virtuales.
- Escribir tests automatizados (tests públicos dados + tests propios en `test_private`).

Requisitos técnicos: **Docker recomendado** para desarrollo y para ejecutar la API y el grading; si no usás Docker, necesitás **Python 3.10+** (compatible con Ubuntu 22.04), entorno virtual (venv) y `pip install -r requirements.txt`. **No se usa Postman**; las pruebas manuales se hacen con `curl`.

1.2 Objetivos de implementación

Codificar una API REST con Flask que cumpla el contrato en `openapi.yaml`. La API debe permitir, en particular:

- Gestionar usuarios (crear, listar, obtener, actualizar, eliminar) y la lista de juegos de cada usuario (agregar, listar, actualizar, eliminar ítems; filtrar por género y ordenar).
- Buscar videojuegos en Wikidata e incorporarlos al catálogo local; poder agregar esos juegos a la lista de un usuario.

Ejemplos de flujo que deberían funcionar correctamente:

1. Crear un usuario con `POST /usuarios`, listarlo con `GET /usuarios`, y obtenerlo con `GET /usuarios/<id>`.
2. Buscar juegos con `GET /juegos?q=zelda&fuente=wikidata`, luego agregar uno a la lista de un usuario con `POST /usuarios/<id>/juegos` (body con `juego_id` como Q-id), y listar la lista con `GET /usuarios/<id>/juegos`.

1.2.1 Si no conocés Python, tests o Docker

Este laboratorio usa **Python**, **pytest** (tests automatizados), **entorno virtual (venv)** y **Docker**. El venv aísla las dependencias del proyecto; el script `make grade` ejecuta los tests, mide cobertura de código, revisa estilo con un linter (ruff) y comprueba complejidad ciclomática. Conviene ver primero los materiales enlazados en “**Recursos por tema**” más abajo. Para ejecutar la API y verificar que todo pase sin instalar Python localmente, se recomienda usar **Docker** (`make docker-run` para la API y `make docker-grade` para el grading). Docker no es obligatorio para aprobar, pero sí recomendado para tener una experiencia cercana a la industria (contenedores, Dockerfile, etc.).

1.3 API para gestión de información

En esta actividad vamos a crear una API robusta utilizando Python y el framework Flask. El proyecto se divide en varias partes: comenzamos con **ejercicios de curl** para familiarizarnos con una API externa (Wikidata). Luego **configuramos el entorno**, levantamos la API y revisamos el **contrato en Swagger** (openapi) y los primeros tests. Después implementamos la API principal para administrar usuarios y sus listas de videojuegos (con campos como “tengo”, “quiero”, “jugado”, “me

gusta”). A continuación integramos nuestra API con Wikidata para buscar videojuegos y enriquecer el catálogo. Por último, evaluamos la API con tests automatizados, cobertura y lint; la entrega incluye el código en un repositorio, evidencia con curl y un video que reemplaza la defensa oral, en el que contarán qué aprendieron, qué dificultades tuvieron y qué herramientas utilizaron.

1.4 Material provisto (kickstarter)

Se entrega un proyecto base (kickstarter) que incluye:

- **openapi.yaml**: contrato formal de la API.
- **Esqueletos en src/**: módulos con stubs que devuelven 501 o respuestas vacías; deben completarse según el contrato. Incluye `app.py`, `store.py`, `usuarios.py`, `juegos.py`, `sugerencias.py`, `wikidata.py`.
- **Tests**: `tests/test_public/` con tests completos que la implementación debe hacer pasar; `tests/test_private/` con esqueletos (TODOs) que deben implementar los estudiantes.
- **grade.py**: script que ejecuta tests, cobertura y lint (`make grade`).
- **Makefile**, **requirements.txt**, **.env.example**, **Dockerfile** y **docker-compose.yml** para ejecutar y desarrollar con Docker (recomendado).

El resto del código debe completarse siguiendo las Partes de este enunciado y el contrato en `openapi.yaml`.

1.5 Antes de empezar: conceptos de red

Glosario

- **Recurso**: entidad identificada por una URL (p. ej. un usuario, una lista de juegos). En REST, las operaciones se aplican sobre recursos.
- **Endpoint**: URL (ruta) que expone una operación de la API; típicamente un método HTTP sobre un path (ej. `GET /usuarios`).
- **Idempotencia**: una operación es idempotente si ejecutarla varias veces produce el mismo resultado que una vez (ej. `GET` no cambia estado; `PUT` de un mismo cuerpo suele ser idempotente).
- **Contrato de API**: especificación (aquí, OpenAPI) que define métodos, rutas, cuerpos y respuestas esperadas.
- **Proxy (en este lab)**: nuestra API actúa como cliente frente a Wikidata y como servidor frente al usuario; ante fallos de la API externa devolvemos 502 (Bad Gateway).

Conceptos que usamos: HTTP (mensajes request/response, métodos, códigos de estado), REST (recursos por URL, API sin estado), OpenAPI como contrato (`openapi.yaml`; Swagger UI en `/docs`). Para búsquedas en Wikidata, el cliente habla con nuestra API y esta consulta a Wikidata; si Wikidata falla, devolvemos 502.

1.5.1 Conceptos de calidad de código (para este lab)

- **Cobertura**: porcentaje de líneas del código en `src/` que son ejecutadas por los tests. Se exige un mínimo (70%) para poder confiar en que los tests ejercitan el código. Si no se alcanza, hay que agregar o mejorar tests.

- **Complejidad ciclomática:** medida del número de caminos o ramas que puede seguir una función (condicionales, bucles, etc.). Las funciones con valor alto son más difíciles de mantener y testear. En este lab **ninguna función en src/ debe superar 8**; si radon marca más, hay que refactorizar (extraer funciones auxiliares, simplificar condicionales).
- **Lintor (ruff):** herramienta que revisa el código sin ejecutarlo (estilo, posibles errores, buenas prácticas). Debe quedar sin errores; los avisos se corrigen en el código.
- Los tests en **tests/test_metrics.py** comprueban automáticamente ruff y complejidad. Si fallan, hay que corregir el código en **src/**, no modificar esos tests.
- **Logging mínimo por request:** la API debe escribir al menos, por cada request, el método, path, código de respuesta y tiempo de respuesta (ms). El kickstarter ya incluye hooks **before_request/after_request** en **app.py** que podés reutilizar o adaptar; no los elimines.

1.6 Contrato de la API

El contrato formal está en **openapi.yaml** en la raíz del proyecto. Con la API en marcha (**make run**), la documentación interactiva está en **http://localhost:5000/docs**. Resumen:

Método	Ruta	Descripción
GET	/health	Health check (Docker, monitoreo).
GET	/usuarios	Listar usuarios
POST	/usuarios	Crear usuario (body: nombre)
GET	/usuarios/<id>	Obtener usuario
PUT	/usuarios/<id>	Actualizar usuario
DELETE	/usuarios/<id>	Eliminar usuario
GET	/usuarios/<id>/juegos	Listar juegos del usuario (query: genero , ordenar = nombre, fecha_lanzamiento o fecha_agregado; orden = asc o desc).
POST	/usuarios/<id>/juegos	Agregar juego a la lista (body: ver OpenAPI). Si ya está: 409 Conflict.
PUT	/usuarios/<id>/juegos/<juego_id>	Actualizar juego (tengo, quiero, jugado, me_gusta)
DELETE	/usuarios/<id>/juegos/<juego_id>	Eliminar juego de la lista
GET	/usuarios/<id>/sugerencias	Sugerencia random entre “juegos que tengo” (query opcional: genero)
GET	/juegos	Listar catálogo; con q busca en local o Wikidata (fuentes=wikidata); query genero , ordenar = nombre, lanzamiento, genero o id; orden = asc o desc.
GET	/juegos/<id>	Obtener juego por id

Cada ítem de lista tiene: **id**, **nombre**, **genero**, **lanzamiento**, **plataforma**, **descripcion**, **tengo**, **quiero**, **jugado**, **me_gusta**, **fecha_agregado**.

1.6.1 Relación entre esquemas de OpenAPI y estructuras en código

Para este lab es importante conectar lo que se ve en Swagger con lo que se implementa en código:

- **Juego** (`components.schemas.Juego`): representa un juego del catálogo global (poblado desde Wikidata). Es el formato que devuelve `GET /juegos` y `GET /juegos/<id>`. En código, estos juegos se guardan en `store.CATALOGO_JUEGOS`.
- **ItemLista** (`components.schemas.ItemLista`): representa un ítem de la lista de un usuario. Combina datos de un Juego del catálogo con los flags del usuario (`tengo`, `quiero`, `jugado`, `me_gusta`) y `fecha_agregado`. En código, se construye a partir de `store.LISTAS_JUEGOS` y `store.CATALOGO_JUEGOS` (podés mirar `solution/src/juegos.py` como ejemplo de referencia).
- **AgregarAListaInput**: es el cuerpo JSON que envía el cliente para agregar un juego a la lista (`POST /usuarios/<id>/juegos`). No representa la lista completa, sino solo el ítem a crear (campos `juego_id`, `tengo`, `quiero`, `jugado`, `me_gusta`).

Sugerencia de lectura: mirá en Swagger los detalles de estos tres esquemas y luego buscá en el código (`store.py`, `juegos.py`) cómo se representan y usan internamente.

Códigos HTTP: 200 (éxito), 201 (recurso creado), 400 (datos inválidos), 404 (no encontrado), 409 (conflicto), 502 (error del servicio externo). Los GET no modifican estado; POST a la lista no es idempotente: agregar el mismo juego dos veces devuelve 409 en la segunda.

1.7 Archivos a completar o modificar

Archivo	Descripción
<code>src/usuarios.py</code>	CRUD de usuarios (crear, listar, obtener, actualizar, eliminar).
<code>src/juegos.py</code>	Lista de juegos por usuario: agregar, listar, actualizar, eliminar ítems; filtro por género y orden.
<code>src/sugerencias.py</code>	Lógica de sugerencia aleatoria (entre juegos que tengo, opcional por género).
<code>src/wikidata.py</code>	Búsqueda en Wikidata, mapeo a esquema de juego, población del catálogo; <code>GET /juegos/<id></code> desde catálogo o Wikidata.
<code>src/store.py</code>	Ya incluye persistencia en SQLite (usuarios, listas, catálogo); no hay que implementarla. Para limpiar datos: <code>make clean-db</code> .
<code>src/filtros.py</code>	Función compartida de filtrado por género y ordenación (asc/desc) para listados de juegos.
<code>src/app.py</code>	Registro de rutas y configuración de la app Flask (el <code>kickstarter</code> ya define las rutas; hay que conectar con los módulos anteriores).
<code>tests/test_private/*.py</code>	Completar los tests privados (casos borde, filtros, sugerencia por género) según los TODOs.

El contrato en `openapi.yaml` no se modifica; la implementación debe cumplirlo.

1.7.1 Sugerencia de reparto del trabajo (grupos de 4)

Rol / responsable	Tareas sugeridas
(1) Usuarios y store	CRUD de usuarios (<code>src/usuarios.py</code>), estructuras en memoria (<code>src/store.py</code>).
(2) Lista de juegos y filtros	Lista de juegos por usuario (<code>src/juegos.py</code>), función compartida de filtrado (<code>src/filtros.py</code>).
(3) Sugerencias	Lógica de sugerencia aleatoria (<code>src/sugerencias.py</code>).
(4) Wikidata e integración	Búsqueda y mapeo Wikidata (<code>src/wikidata.py</code>), registro de rutas en <code>app.py</code> y pruebas de integración.

Notas: El archivo `src/filtros.py` es compartido; conviene acordar su interfaz (parámetros, formato de salida) pronto. En `app.py` puede trabajar quien integre o repartir por rutas. Los tests privados y el video pueden repartirse (quién escribe tests, quién edita el video, quién presenta cada bloque en la grabación). La entrega parcial (Partes 1–3) sirve para que los docentes vean el avance y puedan acercarse si hace falta.

1.8 Parte 1: Jugando con APIs (curl)

Antes de implementar, familiarizate con APIs usando **curl**. La API de Wikidata no requiere API key; se recomienda enviar un User-Agent (por ejemplo `LabRedes2026/1.0`).

1.8.1 Ejemplo 1 — Búsqueda de entidades en Wikidata

```
curl -A "LabRedes2026/1.0" "https://www.wikidata.org/w/api.php?action=wbsearchentities&search
```

- Método: GET. Parámetros: `action=wbsearchentities`, `search`, `language`, `format=json`.
- El header `-A` (User-Agent) identifica tu cliente; Wikidata recomienda enviarlo.

Para leer mejor la respuesta JSON, podés formatearla con Python:

```
curl -A "LabRedes2026/1.0" "https://www.wikidata.org/w/api.php?action=wbsearchentities&search  
| python3 -m json.tool
```

Preguntas guía: - ¿Qué valor aparece en la clave `searchinfo.search`? - En la lista `search`, elegí un resultado que parezca un videojuego: ¿cuál es su id (Q-id), `label` y `description`?

1.8.2 Ejemplo 2 — Obtener una entidad por ID

A partir de un id que obtengas en la respuesta del ejemplo anterior (por ejemplo `Q12395`):

```
curl -A "LabRedes2026/1.0" "https://www.wikidata.org/w/api.php?action=wbgetentities&ids=Q1239
```

- Parámetros: `ids` (uno o más Q-ids separados por `|`), `props` para elegir qué datos traer (labels, descriptions, claims, etc.).

Podés formatear también esta respuesta con:

```
curl -A "LabRedes2026/1.0" "https://www.wikidata.org/w/api.php?action=wbgetentities&ids=Q1239" | python3 -m json.tool
```

Preguntas guía: - Dentro de `entities.<QID>.labels`, ¿qué value aparece para el idioma `es` y para `en`? - Dentro de `entities.<QID>.descriptions`, ¿qué descripción en `es` te devuelve para ese juego?

1.8.3 Ejemplo 3 — Búsqueda con otro término y límite

```
curl -A "LabRedes2026/1.0" "https://www.wikidata.org/w/api.php?action=wbsearchentities&search"
```

- Podés probar distintos valores de `search`, `language` y `limit`.

Si querés ver claramente los 5 primeros resultados, usá:

```
curl -A "LabRedes2026/1.0" "https://www.wikidata.org/w/api.php?action=wbsearchentities&search" | python3 -m json.tool
```

Preguntas guía: - ¿Cuántos elementos hay en la lista `search` cuando usás `limit=5`? - ¿Encontrás en los resultados algún videojuego de la saga Mario? ¿Cuál es su `id` y su `description`?

1.8.4 Consignas para que redactes vos

- En el mismo documento, redactá **al menos un ejercicio adicional** que proponga una consigna, el comando `curl` y un ejemplo de respuesta relevante (podés basarte en la [documentación de la API de Wikidata](#)).
- Incluí al menos un ejercicio que use un endpoint o parámetro distinto a los de los ejemplos (por ejemplo otro idioma, más resultados, o otra acción de la API).

Entregable (Parte 1): documento (Markdown o texto) con los tres ejercicios de ejemplo más los que agregues; en cada uno: consigna, comando `curl` y ejemplo de respuesta relevante. Opcional: tag `fase-0` en el repo.

1.9 Parte 2: Configuración del entorno, Swagger y primeros tests

1. Descargá el repositorio del laboratorio o creá uno con git. Incluí un `.gitignore` adecuado para Python (por ejemplo el [Python.gitignore](#)).
2. Creá un entorno virtual con **Python 3.10** (o superior):

```
python3 -m venv .venv
```

3. Activá el entorno virtual:

```
source .venv/bin/activate # En Windows: .venv\Scripts\activate
```

4. Con el `venv` activo, instalá las dependencias:

```
pip install -r requirements.txt
```

5. Copiá `.env.example` a `.env` para configurar variables de entorno (opcional: `WIKIDATA_USER_AGENT` para las llamadas a Wikidata).
6. **Desarrollo con Docker (recomendado):** Podés usar `make docker-build` y `make docker-run` para levantar la API sin instalar Python localmente. Para verificar que el código cumple todos los criterios (tests, cobertura, lint, complejidad), ejecutá `make docker-grade` desde la raíz del proyecto: corre el grading dentro de un contenedor. El **Dockerfile** está en la raíz; con `docker compose up` podés tener la API con recarga automática si está configurado (p. ej. `FLASK_DEBUG=1`).
7. **Levantar la API y abrir Swagger:** Levantá la API con `make run` (o `make docker-run`). Con la API corriendo, abrí <http://localhost:5000/docs> (Swagger UI). Ahí podés leer el **contrato** de la API (basado en `openapi.yaml`): qué endpoints existen, qué parámetros y body pide cada uno. Probá desde Swagger (o con curl) algunos endpoints; aunque por ahora respondan 501 o vacío, sirve para ver qué tenés que implementar después.
8. **Correr tests e interpretar errores:** Ejecutá `make test` (o `make grade`). Van a fallar varios tests porque la implementación aún no está hecha. Leé la salida: qué test falla, en qué archivo y línea, y qué assertion no se cumple. El video de pytest en “Recursos por tema” explica cómo leer el stack trace. Opcional: anotar en el README o en un documento qué errores aparecieron la primera vez y qué significan.
9. **Prueba con pytest:** Al correr `make test` usá el mensaje de error (archivo, línea, assertion) para entender qué espera cada test; así vas conociendo la herramienta de tests que usarás durante todo el proyecto.

1.10 Parte 3: API de usuarios

Implementar CRUD de usuarios según el contrato.

- Crear, listar, obtener, actualizar y eliminar usuarios.
- En crear/actualizar: **nombre** (obligatorio en creación).

En el proyecto base (`kickstarter/src/usuarios.py`) el endpoint `POST /usuarios` (función `crear_usuario`) ya viene **implementado como ejemplo** completo: valida el body (**nombre**), usa `store.USUARIOS` y `store.LISTAS_JUEGOS`, llama a `next_usuario_id()` y persiste los cambios. Te recomendamos leer esa función junto con el contrato en `openapi.yaml` y los tests públicos de usuarios para tomarla como modelo al implementar el resto de los endpoints de este módulo.

Entregable: endpoints funcionando; evidencia con comandos curl en el documento de ejercicios o en un archivo aparte.

1.11 Parte 4: Lista de juegos por usuario

Cada usuario tiene una única lista de ítems. Cada ítem es un juego con los booleanos: **tengo**, **quiero**, **jugado**, **me_gusta**.

- Agregar, listar, actualizar y eliminar ítems de la lista del usuario.
- En `GET /usuarios/<id>/juegos`: soportar query **genero** (filtrar por género), **ordenar** (**nombre**, **fecha_lanzamiento**, **fecha_agregado**) y **orden** (**asc** | **desc**) para la dirección de ordenación. Podés centralizar la lógica de filtrado y ordenación en el módulo `filtros.py`.

Para implementar `src/juegos.py`, reutilizá la misma idea que en `crear_usuario` de `src/usuarios.py`: ahí se ve cómo armar el diccionario del recurso, actualizar las estructuras de `store` (`USUARIOS`, `LISTAS_JUEGOS`) y llamar a las funciones de persistencia antes de devolver la respuesta. En la lista de juegos vas a trabajar con `store.LISTAS_JUEGOS[usuario_id]` (los ítems del usuario) y `store.CATALOGO_JUEGOS` (datos completos del juego, esquema `ItemLista` de `openapi.yaml`).

Entregable: tag opcional fase-2 en el repo.

1.12 Parte 5: Integración con Wikidata

1.12.1 Documentación de la API de Wikidata

La API de Wikidata expone varios módulos vía <https://www.wikidata.org/w/api.php>. Para este lab son relevantes:

- **Búsqueda por texto:** `action=wbsearchentities` — parámetros `search`, `language`, `limit`; la respuesta incluye una clave `search` con ítems que tienen `id` (Q-id).
- **Obtener entidades por id:** `action=wbgetentities` — parámetros `ids` (varios separados por |), `props` (p. ej. `labels`, `descriptions`, `claims`), `languages`. La respuesta tiene `entities`; cada entidad puede tener `labels`, `descriptions` y `claims`. Los `claims` tienen estructura anidada: `mainsnak` → `datavalue` → `value`, con `id` (para referencias a otra entidad) o `time` (para fechas).

Propiedades e ítem usados en el lab: **P31** (instance of), **Q7889** (video game), **P577** (publication date), **P136** (genre), **P400** (platform). Para filtrar videojuegos se comprueba que la entidad tenga `P31=Q7889`; para el esquema de juego se extraen `P577` (fecha), `P136` (género) y `P400` (plataforma) desde la estructura de `claims`.

- Implementar (o completar) `GET /juegos?q=<texto>&fuente=wikidata`: cuando `fuente=wikidata`, llamar a la API de Wikidata, filtrar videojuegos (`P31=Q7889`), mapear al esquema (`id`, `nombre`, `genero`, `lanzamiento`, `plataforma`, `descripcion`) y actualizar el catálogo local. Sin `q`: listar todo el catálogo. Con `q` y `fuente=local` (o sin `fuente`): buscar en el catálogo por nombre.
- En `GET /juegos` soportar además los query **genero** (filtrar por género), **ordenar** (**nombre**, **lanzamiento**, **genero**, **id**) y **orden** (**asc** | **desc**) para filtrar y ordenar la lista devuelta (ascendente o descendente). La lógica puede reutilizar el módulo `filtros.py`.
- Implementar `GET /juegos/<id>`: devolver un juego por `id` (desde catálogo o desde Wikidata si no está). Los `ids` son strings (Q-id, ej. `Q12395`).

- Opcional: configurar `WIKIDATA_USER_AGENT` en `.env`.
- Los usuarios podrán agregar a su lista juegos obtenidos desde Wikidata usando `POST /usuarios/<id>/juegos` con `juego_id` como string (Q-id).

En todos los casos, las llamadas a la API de Wikidata deben usar un **timeout explícito** (no pueden bloquear indefinidamente). Si la búsqueda `GET /juegos?q=...&fuente=wikidata` falla (timeout, error HTTP, respuesta inválida), la API debe devolver **502 Bad Gateway**. Para `GET /juegos/<id>` y `POST /usuarios/<id>/juegos`, si no se puede obtener ese juego en particular desde Wikidata (porque la entidad no existe o hay un error al resolverla), se devuelve **404 Juego no encontrado** (no 502), ya que el efecto visible para el cliente es que ese id no está disponible.

Entregable: flujo “buscar en Wikidata (`GET /juegos?q=X&fuente=wikidata`) y agregar a mi lista” funcionando, con filtro y ordenación operativos en ambos endpoints de listado.

Pensá `store.CATALOGO_JUEGOS` como el “USUARIOS” de los juegos: un diccionario `id → juego` con el formato del esquema `Juego` de `openapi.yaml`. Las funciones de `src/wikidata.py` deberían leer y escribir en `CATALOGO_JUEGOS` (igual que `src/usuarios.py` hace con `USUARIOS`) y luego devolver los datos ya normalizados.

1.13 Parte 6: Autenticación básica

En el **proyecto base (kickstarter)** la autenticación **ya está implementada y cableada**: existen `POST /auth/registro` y `POST /auth/login`, las rutas en `app.py`, el módulo `src/auth.py` (versión deliberadamente **insegura** para que la corrijas), helpers en `src/auth_common.py` y la tabla **credenciales** en SQLite vía `store.py`. **No tenés que diseñar el flujo desde cero**: tu trabajo es **endurecer la seguridad** hasta cumplir el contrato en `openapi.yaml` y los tests.

El objetivo NO es un sistema de producción completo, sino conceptos mínimos que vas a **aplicar en código** sobre todo en `auth.py`:

1. **Hash de contraseña:** dejar de guardar la contraseña en claro en `password_hash`; al registrar, persistir un **hash** con una función estándar (por ejemplo `werkzeug.security.generate_password_hash`); al hacer login, verificar con la función pareja (`check_password_hash` o equivalente), sin comparar texto plano.
2. **Expiración del token:** al hacer login, generar el token opaco y guardar `token_expira_en` con un **TTL** por defecto razonable (por ejemplo **1 hora**), pudiendo parametrizarse con variable de entorno como referencia en el propio `kickstarter`.
3. **Token vencido o inválido:** donde se interpreta el header `Authorization: Token <TOKEN>`, el token debe considerarse **inválido** si no existe, no tiene expiración coherente o `token_expira_en` es anterior a “ahora”; en esos casos las operaciones que exigen identidad válida deben responder **401** según lo que comprueban los tests públicos.

1.13.1 Modelo de datos (referencia)

- Cada usuario sigue con `id` y `nombre`; el registro por auth agrega **username** único.
- Tabla **credenciales** (u equivalente en `store`): `usuario_id`, `username` (UNIQUE), `password_hash`, `token`, `token_expira_en`.

1.13.2 Uso del token y autorización mínima

- Header: `Authorization: Token <TOKEN>` (el contrato y Swagger lo documentan con el esquema de seguridad del OpenAPI).
- **Autorización mínima:** el token prueba “soy el usuario <id>”; no hay roles.
 - `PUT /usuarios/<id>` y `DELETE /usuarios/<id>`: sólo si el token corresponde a ese <id>; si no → **401**.
 - `POST / PUT / DELETE` sobre `/usuarios/<id>/juegos...`: sólo si el token es del mismo <id>; si no → **401**.
 - `GET /usuarios/<id>/juegos`: seguí el comportamiento descrito en `openapi.yaml` y los **tests públicos** de juegos. En esta edición **no** se pide modelo de lista pública/privada; una extensión posible para el futuro está anotada en el roadmap técnico del repositorio del laboratorio (docentes).

Reutilizá los helpers de `auth_common.py` (parseo de bodies, unicidad de `username`, extracción del header) para no duplicar lógica. La referencia de comportamiento fino sigue siendo `tests/test_public/test_api_auth.py` y el OpenAPI (incluido `/auth/registro` y `/auth/login`).

1.13.3 Relación con la aprobación

- Esta parte es **obligatoria**: la corrección automática (tests públicos + `grade.py`) comprobará, al menos:
 - Que las contraseñas no se guardan en texto plano (se usa un `password_hash`).
 - Que existe un endpoint de login que devuelve un token.
 - Que el token tiene una expiración y los tokens vencidos son rechazados.
 - Que un usuario no puede modificar/borrar otro usuario ni cambiar la lista de juegos de otro.
 - En el README, agregá un breve apartado explicando:
 - Cómo implementaste el hashing de contraseña (qué librería usaste).
 - Cómo funciona la expiración de tokens.
 - Cómo probar login y uso del token con `curl`.
-

1.14 Parte 7: Evaluación de la API y entrega final

1. Ejecutá los **tests públicos** (`tests/test_public/`) y asegurate de que pasen.
2. **Implementá tests propios** en `tests/test_private/` (casos borde, filtros, sugerencia por género). Los archivos de `tests/test_private/` no pueden quedar con `pytest.skip`: al menos los tests indicados en cada archivo deben estar implementados. El script `grade.py` ejecuta tests públicos y privados.
3. Ejecutá `make grade` (o `make docker-grade`) y cumplí cobertura mínima, lint y complejidad. El script también comprueba complejidad ciclomática (límite 8 por función); si falla, revisá la salida de radon y refactorizá (dividí funciones largas o con muchas ramas). **Prueba con ruff**: ejecutá `ruff check src/` (o `make lint`); corregí al menos un aviso o error que muestre para entender para qué sirve el linter. **Prueba con radon**: ejecutá `radon cc src/` para

ver la complejidad ciclomática por función; si alguna supera 8, refactorizá (extraer funciones auxiliares) hasta que `make grade` pase.

4. Recomendado: usá Docker para levantar la API (`make docker-run`) y para verificar con `make docker-grade`.
 5. Grabá el **video** y prepará la **presentación** según las indicaciones más abajo.
 6. **Tag recomendado:** `entrega-final` en el repositorio.
-

1.15 Condiciones de aprobación

Qué comprueba `make grade`: (1) Tests: públicos, `test_private` y `test_metrics`. (2) Cobertura $\geq 70\%$ sobre `src/`. (3) Ruff sin errores en `src/`. (4) Ninguna función en `src/` con complejidad ciclomática > 8 (radon). Si algún ítem falla, el script devuelve error; hay que corregir el código (no modificar los tests de métricas).

- Código que cumpla el contrato en `openapi.yaml` y haga pasar los tests públicos.
 - Tests propios implementados en `tests/test_private/` (los esqueletos completados).
 - **`make grade`** (o `make docker-grade`) debe pasar: todos los tests (públicos y `test_private`), cobertura mínima (70%), sin errores de ruff, y **ninguna función en `src/` con complejidad ciclomática mayor a 8** (el script usa radon; si falla, refactorizar dividiendo funciones).
 - Documento con ejercicios de curl (Parte 1) y evidencia de uso de la API.
 - Implementación de autenticación básica (hash de contraseña + tokens con expiración) según la Parte 6 de este enunciado.
 - README.md con las secciones indicadas más abajo (incluye declaración de uso de IA si aplica).
 - Video y presentación entregados en tiempo y forma, con el contenido y formato indicados.
-

1.16 Entrega

1. Código

- Los archivos del proyecto con las implementaciones realizadas.
- Código que cumpla el contrato en `openapi.yaml`.
- Tests propios implementados en `tests/test_private/` (sin `pytest.skip` en los tests indicados en el enunciado).
- Un documento (txt, md o similar) con los ejercicios de curl (Parte 1) y evidencia de uso de la API con curl (cuando corresponda).

2. README.md (obligatorio)

En la raíz del repositorio debe haber un **README.md** que incluya al menos:

- (a) Instrucciones de compilación y ejecución (Docker recomendado: `make docker-build`, `make docker-run`, `make docker-grade`; o bien entorno virtual, `pip install`, `make run`, `make grade`).
- (b) Breve descripción de los archivos o módulos principales del proyecto.
- (c) Metodología de trabajo en equipo (cómo se repartieron tareas, revisiones, etc.).

- **(d)** Enlace al video y, si aplica, a la presentación (slides) usada en el video.
- **(e) Glosario:** deben armar un glosario con definiciones de los conceptos del laboratorio (por ejemplo: API REST, endpoint, recurso, idempotencia, contrato de API, cobertura de tests, complejidad ciclomática, linters, Docker, health check, timeout, variables de entorno, y los que figuren en el enunciado). Incluir además otros términos que les parezcan relevantes para el proyecto.
- **(f) Preguntas conceptuales y reflexión:** incluir respuestas breves (o un apartado en el README) sobre: qué es una API REST y un contrato de API, para qué sirven los tests y la cobertura, qué es la complejidad ciclomática y por qué se limita, qué es un linter, para qué sirve Docker en este proyecto, y cómo realizaron el debugging cuando algo fallaba.
- **(g) Decisiones de diseño:** qué decisiones de diseño tomaron (por ejemplo estructura de datos, uso de filtros compartidos) y qué problemas encontraron y cómo los resolvieron.
- **(h) Autenticación básica:** un breve apartado explicando cómo implementaron el hashing de contraseñas (librerías usadas), cómo funciona la expiración de tokens y cómo probar registro/login/uso del token con `curl` (resumen de lo indicado en la Parte 6).
- **(i) Uso de IA en el proyecto:** cómo utilizaron asistentes de IA (ChatGPT, Copilot, Cursor, etc.) en el proyecto: en qué partes (código, tests, documentación, debugging), de qué forma (consultas, generación de código, explicaciones) y qué criterios usaron para revisar o adaptar lo generado. Si no usaron IA, indicarlo explícitamente.

3. Video y presentación

- **Link al video** y archivo de la **presentación** (slides) usada en el video. Ver indicaciones detalladas en la sección “Estructura e indicaciones para el video”.
- Uso de la rúbrica de criterios de evaluación (al final de este enunciado) para autoevaluación.

No se utiliza Postman. Las pruebas manuales se realizan con `curl`.

1.16.1 Cómo y cuándo entregar

- **Modalidad:** subir el trabajo a un repositorio (por ejemplo en GitHub/GitLab); la entrega se considera el estado de la rama principal en el **último commit** realizado antes del límite horario.
- **Etiquetas sugeridas:** opcionalmente podés usar tags para marcar hitos: `v0.0-kickstart` (proyecto base), `v0.1-parcial` (entrega parcial, Partes 1 a 3), `entrega-final` (entrega final).
- **Entrega parcial:** se realiza una entrega parcial que debe incluir **Partes 1 a 3** (ejercicios `curl`, configuración del entorno y Swagger, API de usuarios). Tag sugerido: `v0.1-parcial`. Fecha de entrega parcial: la que indique la cátedra en el curso.
- **Entrega final: Lunes 6 de abril a las 23:59** (o la fecha que indique la cátedra). Incluye todo el laboratorio (Partes 1 a 7), README.md, documento de `curl`, video y presentación. Tag recomendado: `entrega-final`.

1.17 Estructura e indicaciones para el video

Duración: aproximadamente 10 minutos (9 a 11 minutos).

Formato exigido: todos los integrantes del grupo deben aparecer en cámara y participar verbalmente

en el video; el video debe reproducirse a **velocidad normal** (sin acelerar). El objetivo es que el grupo dé cuenta del trabajo realizado: reparto de tareas, dificultades, decisiones de diseño y uso de herramientas (no solo describir el proyecto).

Qué mostrar y qué explicar (por bloques):

1. API de usuarios

- **Mostrar:** un flujo con curl o Swagger: crear usuario, listar, obtener por id, actualizar, eliminar (o al menos un subconjunto representativo).
- **Explicar:** cómo modelaron el recurso usuario y qué códigos HTTP devuelve cada operación.

2. Lista de juegos por usuario

- **Mostrar:** agregar un juego a la lista de un usuario, listar con filtro (género) u orden, actualizar un ítem, eliminar.
- **Explicar:** estructura del ítem (tengo, quiero, jugado, me_gusta) y cómo implementaron filtros y orden.

3. Integración con Wikidata

- **Mostrar:** búsqueda con GET `/juegos?q=...&fuente=wikidata`, obtención de un juego por id con GET `/juegos/<id>`, y agregar ese juego a la lista de un usuario.
- **Explicar:** cómo obtienen y mapean los datos de Wikidata al esquema de la API; qué hacen cuando Wikidata no responde (502).

4. Pruebas y calidad

- **Mostrar:** ejecución de `make grade` (o `make test` y `lint`) y que los tests pasen.
- **Explicar:** qué tests propios agregaron en `test_private` y qué cubren (casos borde, filtros, etc.).

Además: qué aprendieron (conceptos de red, REST, OpenAPI), qué dificultades tuvieron y cómo las resolvieron; **cómo se repartieron el trabajo**, qué módulos implementó cada uno, **qué fue lo que más les costó**, **qué decisiones de diseño tomaron** y **qué herramientas utilizaron** (curl, pytest, Flask, git, Docker, etc.).

Estructura opcional: introducción al proyecto; desarrollo por bloques (usuarios, lista, Wikidata, pruebas); conclusiones.

Entregar **link al video** y **archivo de la presentación** (PowerPoint o similar) utilizada en el video.

1.18 Criterios de evaluación (para autoevaluación)

Cada criterio se califica de **0 (Mal)** a **3 (Excelente)**. Para aprobar cada dimensión hay que alcanzar al menos el puntaje mínimo indicado en la tabla de mínimos. La aprobación final requiere cumplir el mínimo en todas las dimensiones exigidas.

1.18.1 Evaluación del código

	1. Endpoints usuarios y Nivellista	2. Filtros, orden, sugerencia	3. Integración Wikidata	4. Git y buenas prácticas	5. Docstrings y tipado	6. Tests (públicos + test_private)	7. Pruebas manuales (curl)	14. README y documentación
Mal (0)	No se implementan CRUD usuarios ni lista de juegos.	No hay filtro por género, orden ni sugerencia.	No se integra Wikidata.	Un solo commit o prácticas deficientes.	Sin docstrings ni tipado.	No hay tests o fallan todos.	No hay evidencia con curl.	README ausente o muy incompleto; no se incluyen secciones clave pedidas en el enunciado.
Regular (1)	algunos endpoints.	Alguna funcionalidad incompleta.	Integración mínima con errores.	Varios commits sin mensajes descriptivos.	Docstrings o tipado parcial.	Algunos tests, incompletos.	Evidencia mínima con curl.	README presente pero superficial: faltan varias secciones obligatorias (por ejemplo glosario, reflexión conceptual o uso de IA) o están apenas mencionadas.

1.	Endpoints	2. Filtros,	3.	4. Git y	5.	6. Tests	7.	14.
	usuarios y	orden,	Integración	buenas	y	(públicos +	manuales	README
Nivellista		sugerencia	Wikidata	prácticas	tipado	test_private)	(curl)	y documentación
Bien	CRUD	Filtro,	Integración	Commits	Docstrings	Tests	Evidencia	README
(2)	usuarios y	orden y	Wikidata	descriptivos	y	públicos	con curl	completo
	lista	sugerencia	funcional.	y	tipado	pasan y	correcta	según el
	según	funcionales.		frecuentes.	en las	test_private	y	enunciado:
	contrato.				funciones.	implementados	replicable.	incluye
								instrucciones,
								descripción
								de
								módulos,
								glosario,
								reflexión
								conceptual,
								decisiones
								de
								diseño,
								uso de
								IA y
								enlaces a
								video/presentación.

1. Endpoints usuarios y Nivelista	2. Filtros, orden, sugerencia	3. Integración Wikidata	4. Git y buenas prácticas	5. Docstrings y tipado	6. Tests (públicos + test_private)	7. Pruebas manuales (curl)	14. README y documentación
Excelente (3)	Implementación robusta. con validaciones.	Integración robusta (ej. 502 cuando falla Wikidata).	Commits semánticos.	Documentación completa.	Tests robustos, casos borde.	Pruebas manuales detalladas.	README muy bien organizado y profundo: explica con claridad decisiones técnicas, incluye ejemplos de uso (por ejemplo comandos curl representativos), describe el flujo de trabajo en equipo y discute aprendizajes y limitaciones del diseño.

1.18.2 Evaluación del video

8. Cobertura Nivel del desarrollo	9. Participación equitativa	10. Estructura y organización	11. Claridad y comunicación	12. Calidad y tiempo	13. Profesionalismo
Mal (0)	No aborda la mayoría de los puntos.	Uno o pocos integrantes presentan.	Sin introducción, desarrollo ni conclusión.	Lenguaje confuso. Calidad deficiente; no respeta 9–11 min.	Actitud poco profesional.

	8. Cobertura Nivel del desarrollo	9. Participación equitativa	10. Estructura y organización	11. Claridad y comunicación	12. Calidad y tiempo	13. Profesionalismo
Regular (1)	Alcoba algunos puntos.	Participación desigual.	Estructura parcial.	Comunicación poco fluida.	Calidad variable.	Profesionalismo parcial.
Bien (2)	Cubre mayoría: entorno, API, integración, pruebas.	La mayoría participa balanceadamente.	Introducción, desarrollo y conclusión claros.	Lenguaje técnico accesible.	Buena calidad; duración en rango.	Actitud profesional.
Excelente (3)	Cubre todo con ejemplos.	Todos participan equitativamente.	Estructura impecable.	Comunicación fluida.	Excelente producción.	Actitud altamente profesional.

1.18.3 Puntajes mínimos para aprobar cada dimensión

Dimensión	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Mínimo	3	3	3	2	2	3	2	2	2	2	2	1	1	2

(Las dimensiones 12 y 13 pueden tener mínimo 0 según criterio de la cátedra.)

1.19 Recomendaciones para el laboratorio

- **Planificá tu trabajo:** dividí el laboratorio en hitos (configuración, curl, API usuarios, lista de juegos, Wikidata, pruebas y video). Hacé autoevaluaciones periódicas.
- **Aprovechá los recursos:** usá el código base del kickstarter, el contrato `openapi.yaml` y la documentación en `/docs`. Consultá la [API de Wikidata](#) cuando integres.
- **Documentá:** comentá y documentá tu código; en el documento de curl dejá consignas y respuestas claras.
- **Prepará los entregables con tiempo:** asegurate de que el código pase `make grade`, que las pruebas sean repetibles y que el video y la presentación sigan las indicaciones.
- **Revisá la rúbrica:** usala para autoevaluarte y cubrir todos los aspectos requeridos.

Consejos de implementación:

- Completá primero el CRUD de usuarios y probalo con curl antes de pasar a la lista de juegos; así tenés un esqueleto estable.
- Integrá por capas: que la lista de juegos use el mismo `store` (o estructuras) que los usuarios, y que Wikidata solo alimente el catálogo sin duplicar lógica.
- Usá los tests públicos como referencia de formato y de casos que debe cumplir la API; cuando implementes `test_private`, pensá en casos borde (id inexistente, lista vacía, filtros sin resultados).

1.20 Mejoras opcionales (puntos estrella)

No son obligatorias; pueden sumar en la evaluación según criterio de la cátedra:

- **Frontend básico para la API:** una pequeña interfaz web (HTML/JS sencillo, o un mini-frontend con el framework que prefieras) que consuma tu API: crear usuarios, listar y filtrar juegos, buscar en /juegos y agregar a la lista, ver sugerencias. La idea es tener un “producto” que puedan mostrar en su portfolio usando la API que implementaron.
- **Segunda fuente de datos externa:** agregar opcionalmente una segunda API de videojuegos además de Wikidata (por ejemplo Steam, RAWG u otra API pública). La idea es integrar una nueva fuente en el flujo de /juegos (por ejemplo agregando un valor extra en **fuentes**) y mapear sus respuestas al mismo esquema de juego que ya usa la API (id, nombre, genero, lanzamiento, plataforma, descripcion), sin romper la funcionalidad principal ni los tests existentes. Esta mejora no se evalúa con tests automáticos del lab, pero puede considerarse como punto adicional en la corrección si está bien documentada y aislada en un módulo propio.
- **Cache / rate limiting sencillo para Wikidata:** guardar en memoria (por ejemplo en un dict) los resultados recientes de búsquedas o juegos por id durante un tiempo acotado, para evitar repetir llamadas idénticas a Wikidata y reducir la carga sobre el servicio externo. También podés limitar la frecuencia de llamadas por segundo. Debe ser una optimización transparente (no cambia las respuestas de la API).
- **Logging más elaborado:** además del logging mínimo obligatorio, agregar un identificador de request (request-id) y/o logs estructurados (por ejemplo en JSON) que faciliten el debugging y el análisis posterior de qué pasó en la API.
- **Tests de integración contra servidor real:** escribir algunos tests marcados, por ejemplo, con `@pytest.mark.integration`, que asuman la API levantada (por ejemplo con `make run` o en Docker) y usen `requests` contra `http://localhost:5000` para probar flujos completos “de caja negra”. Estos tests no se ejecutan en el grading automático salvo que los docentes lo indiquen, pero muestran que saben diferenciar tests unitarios de pruebas end-to-end.

1.21 Cómo ejecutar

Recomendado (Docker):

```
cp .env.example .env          # Opcional: WIKIDATA_USER_AGENT
make docker-build
make docker-run                # Levanta la API en http://localhost:5000
make docker-grade              # Grading (tests + cobertura + lint + complejidad) sin instalar P
```

Para `make docker-run` es necesario tener un archivo `.env` en la raíz (copialo desde `.env.example`). Con la API corriendo, la documentación interactiva (Swagger UI) está en `http://localhost:5000/docs`.

Alternativa sin Docker:

```
python3 -m venv .venv
source .venv/bin/activate      # En Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

```

cp .env.example .env          # Opcional: WIKIDATA_USER_AGENT
make run                      # Levanta la API en http://localhost:5000
make test                    # Ejecuta tests
make grade                   # Grading (tests + cobertura + lint + complejidad)

```

La API escucha en el puerto **5000** en todas las interfaces (0.0.0.0). En producción suele ir detrás de un reverse proxy (HTTPS, otro puerto).

1.22 Recursos por tema

Material de la cátedra y enlaces para apoyar el laboratorio:

Tema	Recurso
APIs (material principal)	Clase grabada de APIs (Drive)
pytest y unit testing (incluye cómo leer el stack trace)	Video pytest (YouTube)
Docker	Video externo Docker (YouTube)
Git y trabajo colaborativo	Presentación Git y colaboración (Google Slides)
Python	Documentación oficial Python 3.10 ; Presentación de clase (YouTube)

1.23 Material de lectura adicional

- [OpenAPI Specification](#) — contrato de API.
- [API de Wikidata](#) — búsqueda y entidades; ayuda de módulos: [wbsearchentities](#), [wbgetentities](#).
- [Flask](#) — framework web usado en el proyecto.
- [pytest](#) — framework de pruebas; [requests-mock](#) para simular HTTP.

1.24 Seguridad y producción (a nivel conceptual)

En este lab se implementa una **autenticación básica** para practicar conceptos de seguridad: hash de contraseñas, tokens con expiración y autorización mínima por usuario. En un entorno de producción, la API tendría requisitos adicionales (rotación de claves, refresh tokens, protección contra ataques de fuerza bruta, etc.). Si se usaran API keys u otros secretos, irían en variables de entorno y **nunca** en el repositorio (`.env` en `.gitignore`). Si más adelante se agrega un frontend en otro origen, será relevante configurar **CORS**.

1.25 Para después

El proyecto incluye persistencia en SQLite en `store.py` (no hay que implementarla; los datos se guardan en un archivo, por defecto `instance/datos.db`). Para volver a empezar con datos vacíos

podés ejecutar `make clean-db`. En un sistema real podría haber varias instancias detrás de un balanceador; el contrato (OpenAPI) seguiría siendo la interfaz entre cliente y servicio.